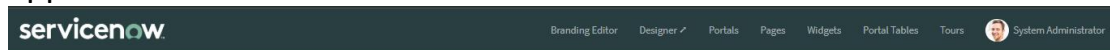


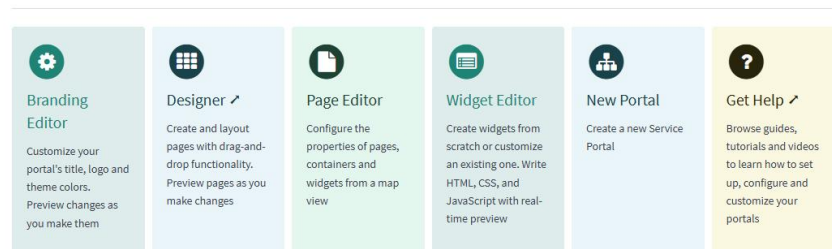
# ServiceNow Service Portal

**Service Portal** is a **web-based, user-friendly interface in ServiceNow** that allows users to access services, submit requests, view information, and interact with ServiceNow applications.



## Service Portal

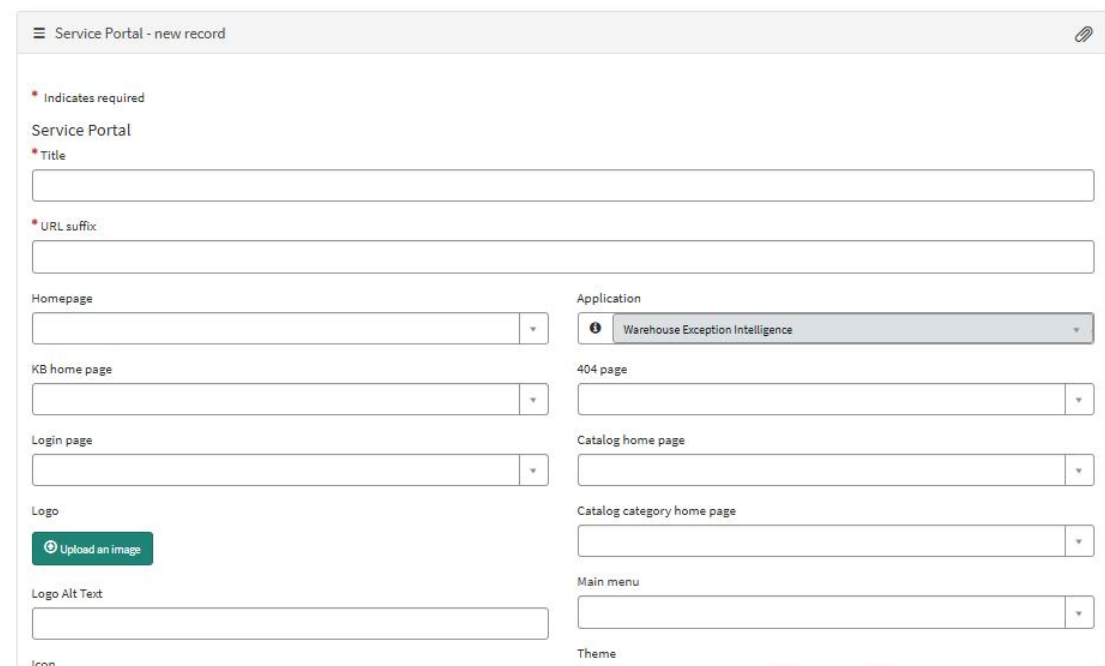
Create rich, engaging and modern experiences to help your business run better  
Select one of the options below to continue



## Portal Configuration

Definition : Portal Configuration defines the overall behavior, appearance, and entry point of a ServiceNow Service Portal.

Navigate to: [Service Portal](#) → [Service portal configurations](#)

A screenshot of the 'Service Portal - new record' configuration form. The form has a header bar with a menu icon and a pencil icon. Below the header, there's a legend: '\* Indicates required'. The form is divided into two columns. The left column contains: 'Service Portal' (title), 'Title' (text input), 'URL suffix' (text input), 'Homepage' (dropdown), 'KB home page' (dropdown), 'Login page' (dropdown), 'Logo' (with an 'Upload an image' button), 'Logo Alt Text' (text input), and 'Icon' (text input). The right column contains: 'Application' (dropdown with 'Warehouse Exception Intelligence' selected), '404 page' (dropdown), 'Catalog home page' (dropdown), 'Catalog category home page' (dropdown), 'Main menu' (dropdown), and 'Theme' (text input).

Think of it as the main configuration of the entire Service Portal.

### **Portal Configuration Fields:**

1. **Title / Name** : Defines the name of the Service Portal.
2. **URL Suffix** : Defines the URL path used to access the portal.
3. **Homepage** : Defines the page that users see when they open the portal.
4. **Logo** : Defines the branding/logo displayed in the portal.
5. **Theme** : Defines the overall visual appearance of the portal.
6. **Header** : The header is the top section of the Service Portal.
7. **Footer** : The footer appears at the bottom of the portal.
8. **Login Page** : Defines the page users see when authentication is required.
9. **Main Menu / Navigation** : The portal can have a navigation menu containing links to different portal pages.

## Service Portal: Pages

A Page in Service Portal represents an individual screen/webpage displayed inside a Service Portal.

A page is built using **containers, rows, columns, and widgets**.

Navigate to: [Service Portal → Pages](#)

The screenshot shows the configuration interface for a Service Portal page. At the top, there's a breadcrumb navigation: "Page" > "x\_1956908\_wareho\_0\_home". A warning message states: "This record has been recently modified in update set Checkpoint by System Administrator but not committed to the source control (View differences). To edit the record use update set Checkpoint or continue in the current update set Checkpoint".

The main configuration area includes:

- Title:** Warehouse Exception Intelligence
- ID:** x\_1956908\_wareho\_0\_home
- Application:** Warehouse Exception Intelligence
- Public:** ☐
- Draft:** ☐
- Roles:** (empty)
- Short description:** (empty)
- Page Specific CSS:** (empty)
- Dynamic page title:** (empty)
- Use SEO script:** ☐
- Human readable url structure:** (empty)

Below the configuration area is the **Page Content** section, which shows the page structure:

- Page: Warehouse Exception Intelligence
- Router
  - Row 100
    - Column 100
      - Instance: (Warehouse Role Router)

There are buttons for "Try It" and "Clone Page". Below this is a "Related Links" section with links to "Open in Designer", "Open in Page Editor", and "Run Point Scan".

At the bottom, there's a "Containers (1)" tab, a "Meta Tags" tab, and a "Dynamic page variables" tab. The "Containers (1)" tab is active, showing a table with the following data:

| Name   | Order | Package                          |
|--------|-------|----------------------------------|
| Router | 100   | Warehouse Exception Intelligence |

The table has a search bar and a "New" button. The page number "1 to 1 of 1" is displayed at the bottom.

From here, we can create and configure Service Portal pages.

### Important Page Components:

1. **Page ID** : A unique identifier used to identify and access the page.
2. **Title** : Defines the name/title of the page.
3. **Description** : Describes the purpose or functionality of the page.

4. **Containers** :The main sections used to organize the content of a page.

5. **Rows** :Used to divide a container into horizontal sections.

6. **Columns** :Used to divide a row into different sections where widgets can be placed.

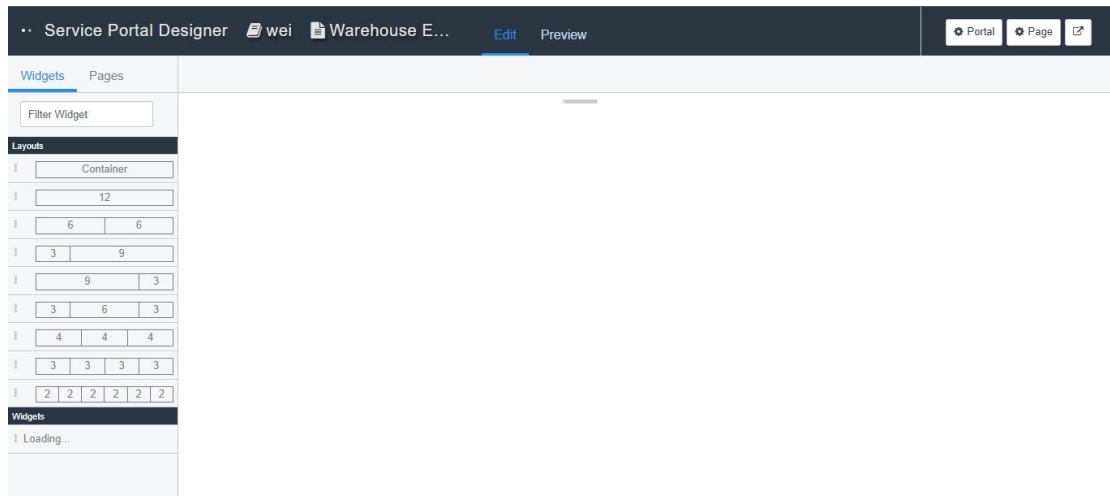
7. **Widgets** :Reusable UI components placed inside columns to provide functionality.

8. **Layout** :Defines how containers, rows, and columns are arranged on the page.

9. **Widget Instances** :A widget placed/configured on a particular page is called a widget instance.

10. **Page Route / URL** :Used to access or map a URL to a particular Service Portal page.

## Pages : Layouts / Containers / Rows / Columns



**Layout** : A Layout defines how the content and widgets are arranged on a Service Portal page. It determines the structure of [containers](#), [rows](#), and [columns](#).

**Container** : A Container is a main section of a Service Portal page that holds one or more rows. It helps organize the overall page structure.

[Page --> Container --> Rows](#)

**Row** : A Row is a horizontal section inside a container. A row can contain one or more columns.

[Container --> Rows --> columns](#)

**Column** : A Column is a section inside a row where widgets can be placed. Columns determine how much horizontal space each widget receives.

[Row](#)

[└─ Column → Widget](#)  
[└─ Column → Widget](#)

## Service Portal : Widgets

A Widget is a **reusable UI component** in Service Portal that provides a specific functionality or displays information on a page.

Ex: **Search Widget, Login Widget, Incident Widget, Knowledge Widget, Catalog Widget**

Navigate to: [Service Portal → Widgets](#)

The screenshot shows the 'Widget New record' form in the Service Portal. The form includes the following fields and controls:

- Name:** A text input field with a red asterisk indicating it is required.
- ID:** A text input field.
- Description:** A large text area.
- Application:** A dropdown menu currently showing 'Warehouse Exception Intelligence'.
- Public:** A checkbox.
- Roles:** A button with a pencil icon for editing roles.
- Body HTML template:** A text area containing the default HTML template: 

```
<div>
<!-- your widget template -->
</div>
```
- CSS:** A large empty text area for custom CSS.

The top of the form has a navigation bar with a back arrow, a menu icon, the title 'Widget New record', and a 'Submit' button.

From here, we can create, configure, and manage Service Portal widgets.

### Main Parts of a Widget:

- 1. HTML Template :** Defines the structure/UI of the widget.Used for: [Text, Buttons, Forms, Tables, HTML elements](#)
- 2. CSS :** Defines the appearance and styling of the widget.Used for: [Colors, Fonts, Size, Spacing, Layout](#)
- 3. Client Controller :** Contains client-side JavaScript that runs in the user's browser.Used for: [Button clicks, User interactions ,Updating the UI, Calling server-side functionality](#)
- 4. Server Script :** Contains server-side JavaScript used to retrieve or process ServiceNow data.Commonly uses: [GlideRecord, \\$sp, Server-side logic](#)

**5. Link Function** : Used for client-side initialization and interacting with the widget's DOM when required.

**6. Option Schema** : Defines configurable options for a widget instance.

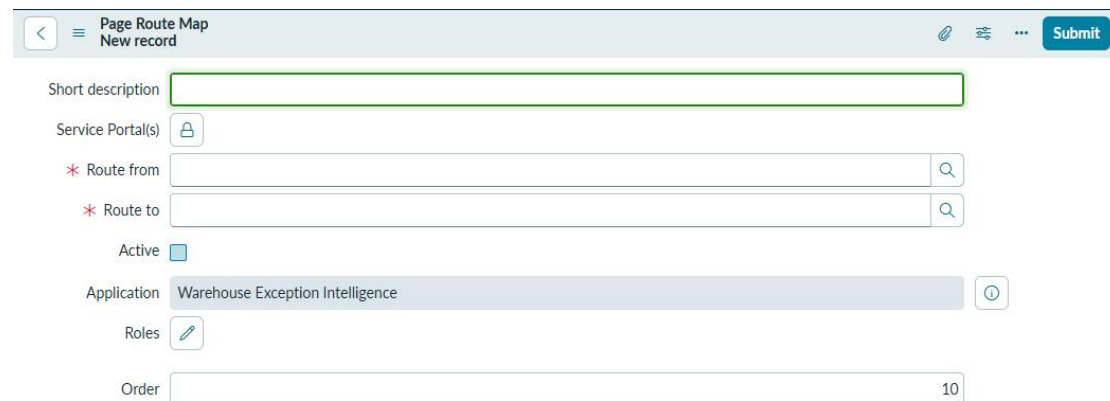
It allows the same widget to behave differently depending on its configuration.

## Service portal : Page Routes Maps

A Page Route defines a **URL route that maps a specific URL/path to a Service Portal page.**

It helps users access pages through friendly and customized URLs.

Navigate to: [Service Portal](#) → [Page Routes](#)



The screenshot shows a 'Page Route Map' form with the following fields:

- Short description**: A text input field.
- Service Portal(s)**: A dropdown menu with a lock icon.
- Route from**: A text input field with a search icon.
- Route to**: A text input field with a search icon.
- Active**: A checkbox.
- Application**: A dropdown menu showing 'Warehouse Exception Intelligence'.
- Roles**: A text input field with a pencil icon.
- Order**: A text input field with the value '10'.

A 'Submit' button is located in the top right corner of the form.

From here, we can create and configure routes for Service Portal pages.

### Important Page Route Fields

**1. Route from:** Defines the URL path used to access the page.

Example: [Route: incident](#)

**2. Route to :** Defines the Service Portal page that should be displayed for that route.

Example: [Route: incident](#) ---> [Page: Incident](#)

**3.Service Portal :** Specifies the Service Portal where the route is available.

**4. Active :** Determines whether the page route is currently active.

**5.Short Description :** Describes the short purpose of this Routes.

## Navigation

Navigation in Service Portal **allows users to move from one page or functionality to another** within the portal.

It is mainly provided through menus and menu items.

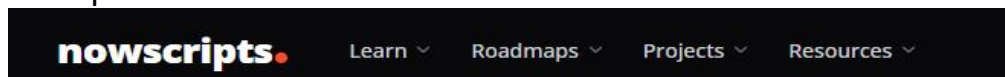
Navigate to: [Service Portal](#) → [Menus](#)

From here, we can create and configure portal navigation menus.

### Important Navigation Components:

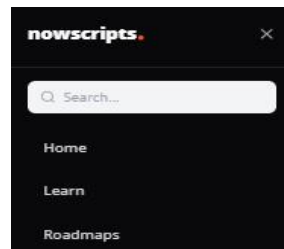
**1. Menu** : A Menu is a collection of navigation items displayed in the Service Portal.

Example:



**2. Menu Item** : A Menu Item is an individual link/action inside a menu.

Example:



**2. URL / Page** : A menu item can navigate the user to a specific Service Portal page or URL.

Example: [Catalog](#) --> [Catalog Page](#)

**4. Order** : Defines the order in which menu items appear.

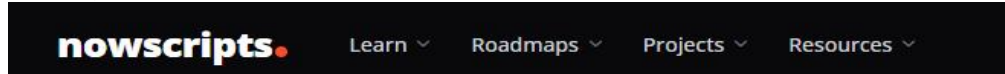
**5. Roles** : Can be used to control which users can see certain menu items.

## Service Portal : Menus

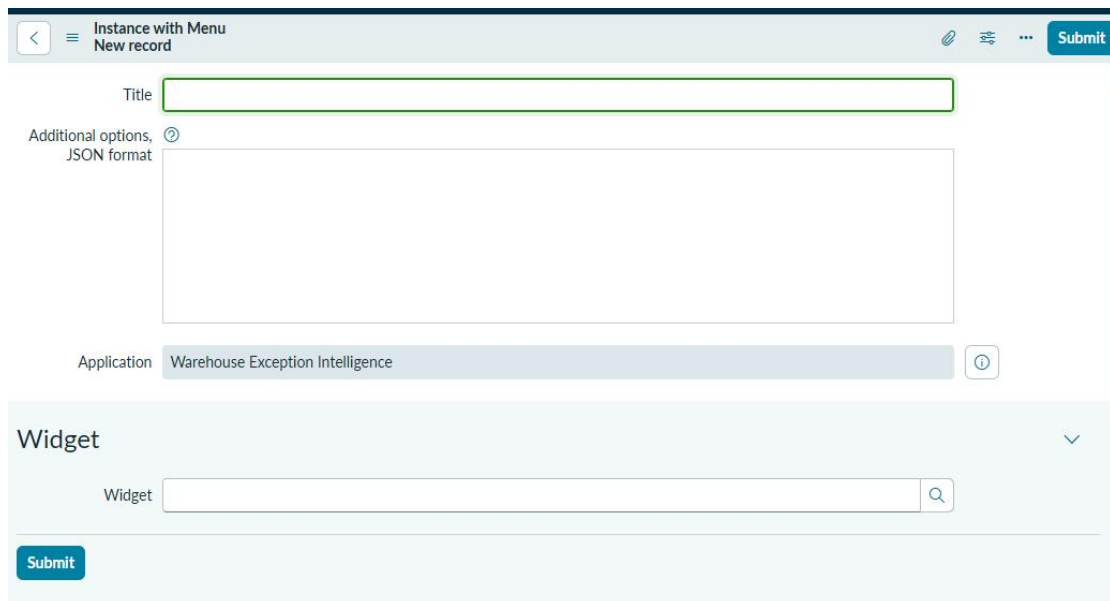
A Menu in Service Portal is a **collection of Menu Items that provides navigation options to users.**

It is commonly displayed in the Service Portal header.

Example:



Navigate to: [Service Portal](#) → [Menus](#)

A screenshot of a web form titled 'Instance with Menu' with a subtitle 'New record'. The form is light blue and contains several input fields. At the top right, there are icons for a link, a list, and a menu, followed by a 'Submit' button. The main form area has a 'Title' field with a green border, an 'Additional options, JSON format' field with a help icon, and an 'Application' dropdown menu currently set to 'Warehouse Exception Intelligence' with an information icon. Below these is a 'Widget' section with a search bar and a 'Submit' button at the bottom left.

From here, we can create and configure Service Portal menus.

### Important Menu Fields:

- 1. Title / Name** : Defines the name of the menu.
- 2. Active** : Determines whether the menu is available for use.
- 3. Menu Items** : Contains the individual navigation options displayed in the menu.

## Service Portal — Themes

A Theme defines the overall look and **visual appearance of a Service Portal**.

It controls the portal's branding and **UI styling, such as colors, fonts, header, and footer**.

Navigate to: [Service Portal → Themes](#)

The screenshot shows the 'Theme' configuration page in a Service Portal. At the top, there's a header bar with 'Theme' and 'New record' on the left, and a search icon on the right. Below the header, the form is organized into two main columns. The left column contains fields for 'Name' (with a green border), 'Turn off SCSS compilation' (a checkbox), 'Logo' (with a 'Click to add...' link), 'Logo Alt Text' (a text input), and 'Icon' (with a 'Click to add...' link). Below these is a 'CSS variables' section with a help icon and a large text area. At the bottom left, there's a 'Matching Next Experience Theme' dropdown. The right column contains 'Application' (a dropdown set to 'Warehouse Exception Intelligence'), 'Header' (a text input with a search icon), 'Footer' (a text input with a search icon), 'Fixed header' (a checked checkbox), and 'Fixed footer' (a checked checkbox). A small information icon is next to the 'Application' dropdown.

From here, we can create and configure themes for Service Portals.

### Important Theme Components:

**1. Header** : Defines the appearance and behavior of the portal's top section.

It can contain: **Logo , Navigation menu, Search , User/profile options**

**2. Footer** : Defines the appearance of the bottom section of the portal.

**3. CSS / Styling** : Controls the visual appearance of portal elements.

Examples: **Colors , Fonts , Backgrounds , Spacing , Buttons**

**4. Logo / Branding** : Used to apply organization-specific branding to the portal.

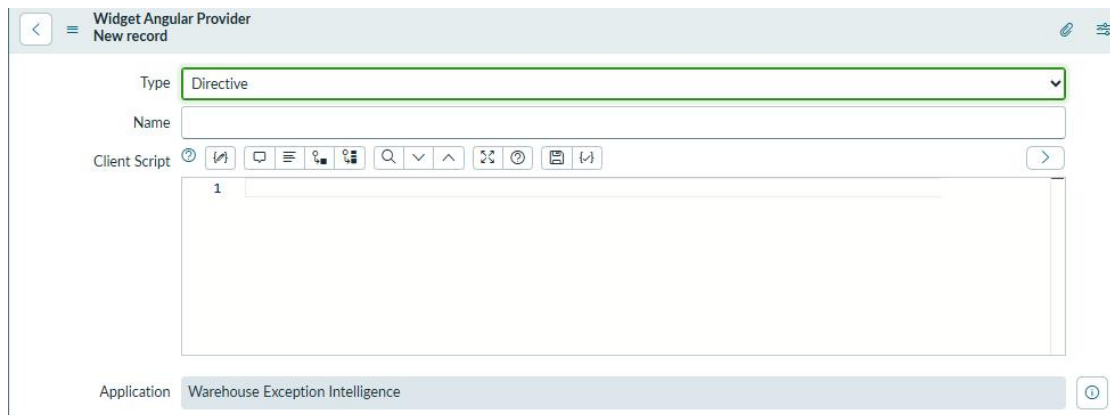
**5. Header & Footer Widgets** : Themes can use configured widgets for displaying custom header and footer content.

## Service Portal — AngularJS

**AngularJS** is a JavaScript framework used by Service Portal to build **dynamic and interactive user interfaces**.

Service Portal widgets use AngularJS for **client-side logic, data binding, events, and UI interactions**.

Navigate to : [Service portal --> Angular Provider](#)



AngularJS is mainly used in the Client Controller of Service Portal widgets.



### Important AngularJS Concepts:

**1. Controller** : Contains client-side logic for a widget.

Used for:

1. Handling button clicks
2. Processing user input
3. Updating UI data
4. Calling server-side logic

**2. \$scope :** Used to share data between the controller and HTML template.

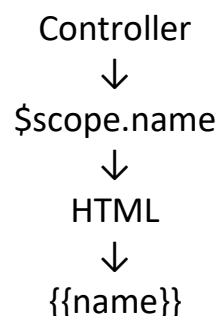
Example: `$scope.name = "Ramu";`

HTML: `{{name}}`

Output: Ramu

**3. Data Binding :** Connects data between the JavaScript controller and HTML.

**Example:**



**4. Expressions :** Used to display dynamic values in HTML.

Example: `{{data.name}}`

**5. Directives :** AngularJS directives extend HTML with additional behavior.

**Common examples:** `ng-if`, `ng-repeat`, `ng-click`, `ng-model`, `ng-show`, `ng-hide`

Example: `<button ng-click="c.submit()">Submit</button>`

**a . ng-model :** Creates two-way data binding between an input field and the model.

Example: `<input ng-model="c.name">`

The value entered by the user is automatically available in the controller.

**b . ng-repeat :** Used to repeat HTML elements for a collection of data.

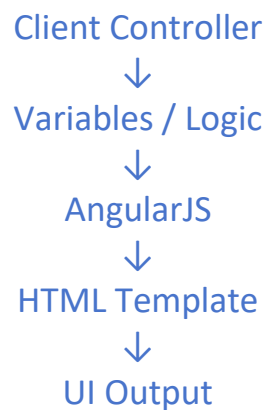
Example: `<div ng-repeat="item in data.items">`  
    `{{item.name}}`  
    `</div>`

**C . ng-if :** Conditionally displays an element.

**Example:**

```
<div ng-if="data.showMessage">
Welcome
</div>
```

**4. Client Controller and HTML :** AngularJS connects the Client Controller with the HTML Template.



**5. AngularJS and Server Script :** AngularJS handles the client side, while the Server Script handles server-side processing.

